

Linux Permissions Practice Question Bank

Basic | Special | Default Formation (umask) | ACLs

Companion to: Linux Permissions Guide (RHEL 10)

What's inside:

- 60 practice questions across 4 sections + a mixed final challenge
- Fill-in-the-blank, command-writing, scenario, and multiple-choice formats
- Difficulty ramps from recall to real sysadmin troubleshooting scenarios
- Full answer key at the end, with brief explanations
- Aligned to RHCSA / RHEL 10 exam-style tasks

How to Use This Question Bank

Work through each section in order — difficulty increases from basic recall to real troubleshooting scenarios. Write your answers on the blank lines or in a terminal, then check the **Answer Key** at the end. Sections mirror the companion guide: Basic Permissions, Special Permissions, Default Formation (umask), and ACLs, followed by a Mixed Challenge section.

SECTION A — BASIC
PERMISSIONS

A. Basic Permissions (Q1–Q15)

Q1. What do the letters r, w, and x mean when applied to a **file** (as opposed to a directory)?

Q2. What does the **execute (x)** permission mean when applied to a **directory** instead of a file?

Q3. Convert this permission string to its 3-digit octal value: `rwxr-xr--`

Q4. Convert the octal value `640` back into its symbolic `rw`x form.

Q5. Write the `chmod` command (symbolic notation) to add execute permission for the group only on a file named `backup.sh`.

Q6. Write the `chmod` command (numeric notation) to set a file's permissions to `rw-rw-r--`.

Q7. Write a single command to give a directory called `/opt/reports` and everything inside it permissions `750`, recursively.

Q8. You run a script and get `-bash: ./deploy.sh: Permission denied`. What is the most likely cause, and which single command fixes it?

Q9. What is the difference between `chown user:group file` and `chgrp group file`?

Q10. Write the command to recursively change both owner and group of `/var/www/app` to `appuser : appgrp`.

Q11. Which command shows you all the groups a user named `asha` belongs to?

Q12. A file shows as `-rw-r-----`. Can a user who is **not** the owner and **not** in the file's group read this file? Explain briefly.

Q13. What command would you use to see a file's exact octal permission mode along with its owner and group, in a single line — useful for scripting?

Q14. Which octal digit represents read + write (no execute)?

- | | |
|------|------|
| A) 4 | B) 6 |
| C) 5 | D) 3 |

Q15. What does `chmod o=r file.txt` do?

- | | |
|--|---|
| A) Adds read for others, keeps existing other bits | B) Sets others' permission to exactly read-only, removing write/execute |
| C) Sets owner's permission to read-only | D) Removes read permission from others |

SECTION B — SPECIAL
PERMISSIONS

B. Special Permissions (Q16–Q30)

Q16. Name the three special permission bits in Linux and give their octal values.

Q17. What does the SUID bit do when set on an executable file?

Q18. Give a real-world example of a RHEL binary that uses SUID, and explain why it needs it.

Q19. Write the chmod command (symbolic) to set SUID on a file called `/usr/local/bin/backup-tool`.

Q20. Write the equivalent command using numeric notation, assuming the base permission should be 755.

Q21. In `ls -l` output, how do you tell the difference between an SUID bit shown as lowercase `s` versus uppercase `S`?

Q22. What does SGID do differently when applied to a **directory** versus a **file**?

Q23. You manage a shared project folder for a 6-person DevOps team. New files created by different team members keep landing with each person's individual primary group instead of the team group. What single command fixes this going forward?

Q24. What is the purpose of the Sticky Bit, and on what type of object (file or directory) is it normally applied?

Q25. Name the one directory that ships with the Sticky Bit set by default on every Linux system, and explain why it needs it.

Q26. Write the numeric chmod command to set **all three** special bits (SUID+SGID+Sticky) on a directory, with base permissions rwxrwxrwx.

Q27. Write a find command that searches the entire filesystem for all files with the SGID bit set, suppressing permission-denied errors.

Q28. Why is it considered a security risk to leave unnecessary SUID binaries on a production RHEL server?

Q29. Which chmod numeric prefix sets the Sticky Bit?

A) 1xxx

B) 2xxx

C) 4xxx

D) 7xxx

Q30. A directory shows drwxrwsr-x. Which special permission is active?

A) SUID

B) SGID

C) Sticky Bit

D) None — this is a typo

SECTION C — DEFAULT
FORMATION (umask)

C. Default Permission Formation (Q31–Q42)

Q31. What are the two baseline permission values that Linux starts from before applying umask — one for new files, one for new directories?

Q32. Explain, in your own words, what umask actually does to those baseline values.

Q33. If umask is set to 022, what permissions will a newly created file get? Show the subtraction.

Q34. If umask is set to 027, what permissions will a newly created **directory** get? Show the subtraction.

Q35. Why do new files never get execute permission by default, even when umask would technically allow it (e.g. umask 000)?

Q36. Which command shows your current umask value in symbolic form (e.g. u=rwx, g=rx, o=rx)?

Q37. You run `umask 027` directly in your terminal. Will this setting still be active after you log out and log back in? Why or why not?

Q38. Name two files you could edit to make a umask change **persistent for a single user only**.

Q39. Name one file you could edit to enforce a umask **system-wide, for all users**.

Q40. How would you set a default umask specifically for files created by a systemd service, independent of any user's shell umask?

Q41. What umask value would you set to ensure new files are never readable by 'other' at all, while group keeps full access? (Show your reasoning.)

Q42. With `umask 077`, what permission will a new directory receive?

- | | |
|--------|--------|
| A) 700 | B) 770 |
| C) 755 | D) 750 |

SECTION D — ACCESS
CONTROL LISTS (ACL)

D. Access Control Lists (Q43–Q52)

Q43. What limitation of standard owner/group/other permissions do ACLs solve?

Q44. Which command lets you **view** all extended ACL entries on a file?

Q45. Write a command to give a user named `rahu1` read+write (but not execute) access to a file called `budget.xlsx` via ACL, without changing its owner.

Q46. Write a command to grant the group `qa` read+execute access to a directory called `/srv/build` via ACL.

Q47. How would you **remove** just the ACL entry you gave to `rahu1` in Q45, without affecting anyone else's access?

Q48. How would you strip **all** extended ACL entries from a file, resetting it back to plain owner/group/other permissions?

Q49. What is a **default ACL** on a directory, and what problem does it solve for files created inside that directory later?

Q50. Write a command that sets a default ACL on `/srv/team` so any new file created inside automatically grants the group `devops` `rwx` access.

Q51. After running `getfacl` on a file, you see an entry starting with `mask::`. What does the ACL mask control?

Q52. Which flag removes a specific ACL entry with `setfacl`?

- | | |
|--------------------|--------------------|
| A) <code>-x</code> | B) <code>-b</code> |
| C) <code>-m</code> | D) <code>-d</code> |

**SECTION E — MIXED
CHALLENGE
(SCENARIOS)**

E. Mixed Challenge — Real Scenarios (Q53–Q60)

These combine multiple concepts, the way an RHCSA task or a real ticket would.

Q53. You need to create a shared directory `/srv/finance` for a finance team where: (1) all team members can read/write files, (2) new files automatically belong to the 'finance' group, and (3) users cannot delete each other's files. Write ALL the commands needed, in order.

Hint: this needs `mkdir`, `chgrp`, and two special permission bits.

Q54. A junior admin ran `chmod 777` on `/etc/passwd` by mistake. What is wrong with this, and what is the correct permission mode to restore?

Q55. You want a specific auditor, `priya`, to have read-only access to a payroll file that is otherwise owned by `hr:hrgroup` with mode `640` — **without** adding her to the `hrgroup` and without changing the file's owner/group. What is the correct approach and command?

Q56. A new hire's home directory has `umask 022`, but company policy requires `umask 027` for all new engineering hires. Where would you make this change so it applies system-wide, and what line would you add?

Q57. You find a suspicious world-writable, SUID-root binary at `/opt/tool/run.sh` during a security audit. What TWO separate risks does this combination create, and what command would you run to remove just the SUID bit (leaving the rest of the permissions unchanged)?

Q58. Explain the full permission 'journey' of a new file created inside a directory that has SGID set AND a default ACL granting group devs rwx access. What group and ACL will the new file end up with?

Q59. Write the single `chmod` command that sets a directory to: SGID on, rwx for owner, rwx for group, r-x for others.

Q60. You're troubleshooting why a user can access a file even though `ls -l` shows permissions that should block them. What is the first command you should run to check for a hidden cause, and what would you be looking for in its output?

ANSWER KEY

Answer Key

Brief answers with reasoning. Compare, don't just copy — try each question yourself first.

Section A — Basic Permissions

- A1.** r = read file contents, w = modify/write file contents, x = execute the file as a program/script.
- A2.** x on a directory allows entering it (cd) and accessing files inside by name / traversing it — without x, even readable file names inside can't be accessed by path.
- A3.** 754 (rwx = 7, r-x = 5, r-- = 4).
- A4.** 640 → rw-r-----
- A5.** chmod g+x backup.sh
- A6.** chmod 664 file
- A7.** chmod -R 750 /opt/reports
- A8.** The file's execute bit isn't set for the user running it. Fix: chmod u+x deploy.sh (or chmod +x deploy.sh).
- A9.** chown can change both owner and group in one command (user:group); chgrp changes only the group, owner stays untouched.
- A10.** chown -R appuser:appgrp /var/www/app
- A11.** groups asha
- A12.** No — rw-r----- gives owner rw, group r, and others nothing (---); a non-owner, non-group user has zero access.
- A13.** stat -c '%a %U %G %n' file.txt
- A14.** 6 (4=read + 2=write)
- A15.** Sets others' permission to exactly read-only, removing any existing write/execute for others (= means 'set exactly', not 'add').

Section B — Special Permissions

- A16.** SUID = 4, SGID = 2, Sticky Bit = 1.
- A17.** The program executes with the privileges of the file's owner, regardless of who runs it.
- A18.** /usr/bin/passwd — it must write to /etc/shadow (owned by root) even when an ordinary user changes their own password.
- A19.** chmod u+s /usr/local/bin/backup-tool
- A20.** chmod 4755 /usr/local/bin/backup-tool
- A21.** Lowercase s means SUID is set AND the owner execute bit is also set; uppercase S means SUID is set but the owner execute bit is NOT set (unusual/broken combination).
- A22.** On a file, SGID runs the process with the file's group privilege. On a directory, SGID makes every new file/subdirectory created inside inherit the directory's group automatically.
- A23.** chmod g+s /path/to/shared/project (SGID on the directory).
- A24.** It prevents users from deleting or renaming files they don't own, even in a directory where they have write access. Applied to directories.

- A25.** /tmp — it's world-writable so any user can create temp files there, but the sticky bit stops one user from deleting another's files.
- A26.** `chmod 7777 dirname` (7 = 4+2+1 for SUID+SGID+Sticky, 777 for rwx all classes).
- A27.** `find / -perm -2000 -type f 2>/dev/null`
- A28.** Any SUID binary is a potential privilege-escalation vector — if it has a vulnerability or is misconfigured, an attacker can use it to gain the owner's (often root's) privileges.
- A29.** 1xxx
- A30.** SGID (lowercase s in the group execute position).

Section C — Default Formation (umask)

- A31.** Files start at 666 (rw-rw-rw-); directories start at 777 (rwxrwxrwx).
- A32.** umask subtracts (masks out) permission bits from the baseline default, defining which permissions are NOT granted automatically to new files/directories.
- A33.** $666 - 022 = 644 \rightarrow rw-r--r--$
- A34.** $777 - 027 = 750 \rightarrow rwxr-x---$
- A35.** The 666 file baseline itself has no execute bit built in — umask can only remove bits that exist in the baseline, it never adds the execute bit that isn't there to begin with.
- A36.** `umask -S`
- A37.** No — a umask set directly in the shell is temporary and only lasts for that session; it resets once you log out.
- A38.** `~/.bashrc` and `~/.bash_profile` (per-user shell startup files).
- A39.** `/etc/bashrc` or `/etc/profile`.
- A40.** Add `UMask=0027` under the [Service] section of the relevant systemd unit file.
- A41.** `umask 037` $\rightarrow$ files: $666-037=640$ (rw-r-----, note x differs on dirs), directories: $777-037=740$. This removes all 'other' access (0 for other) while keeping group at read (and execute for dirs). Exact value depends on desired group rights — 027 keeps group at rx/rw while zeroing other.
- A42.** 700 ($777 - 077 = 700$).

Section D — Access Control Lists

- A43.** Standard permissions only support one owner and one group; ACLs let you grant different permissions to multiple additional users or groups on the same file.
- A44.** `getfacl filename`
- A45.** `setfacl -m u:rahul:rw budget.xlsx`
- A46.** `setfacl -m g:qa:rx /srv/build`
- A47.** `setfacl -x u:rahul budget.xlsx`
- A48.** `setfacl -b filename`
- A49.** A default ACL is an ACL template attached to a directory; any new file or subdirectory created inside automatically inherits those ACL entries, so permissions stay consistent without manual setfacl calls each time.
- A50.** `setfacl -m d:g:devops:rwx /srv/team`
- A51.** The mask defines the maximum effective permissions that named users/groups (via ACL entries) can have, regardless of what's individually listed — it caps the effective ACL permissions.
- A52.** -x

Section E — Mixed Challenge

A53. `mkdir /srv/finance; chgrp finance /srv/finance; chmod 2770 /srv/finance` (2 = SGID for group inheritance, 770 = rwx for owner+group, none for other) — then add +t if 'others' also need access: `chmod 3770` for SGID+Sticky if the directory is shared beyond the group.

A54. 777 makes `/etc/passwd` world-writable, letting any user modify system account data (severe security hole). Correct mode is 644 (`rw-r--r--`), owned by `root:root`.

A55. Use an ACL: `setfacl -m u:priya:r-- payrollfile` — grants priya read-only access individually without touching the group or owner.

A56. Edit `/etc/profile` (or `/etc/bashrc`) and add/modify the line: `umask 027` — ideally scoped with a conditional for the engineering group if needed.

A57. Risks: (1) world-writable means any user can modify/replace the script, (2) SUID-root means that modified script then runs with root privileges — a direct root-compromise path. Fix SUID: `chmod u-s /opt/tool/run.sh`

A58. The new file inherits the SGID directory's group (not the creating user's primary group), AND it inherits the default ACL entry, granting the 'devs' group rwx access as an ACL entry on top of the standard group permission.

A59. `chmod 2775 dirname` (or `chmod g+s,u=rwx,g=rwx,o=rx dirname` symbolically).

A60. Run `getfacl` on the file — look for extended ACL entries (a '+' after the permission string in `ls -l` is the giveaway) that grant access beyond what the standard owner/group/other bits show.